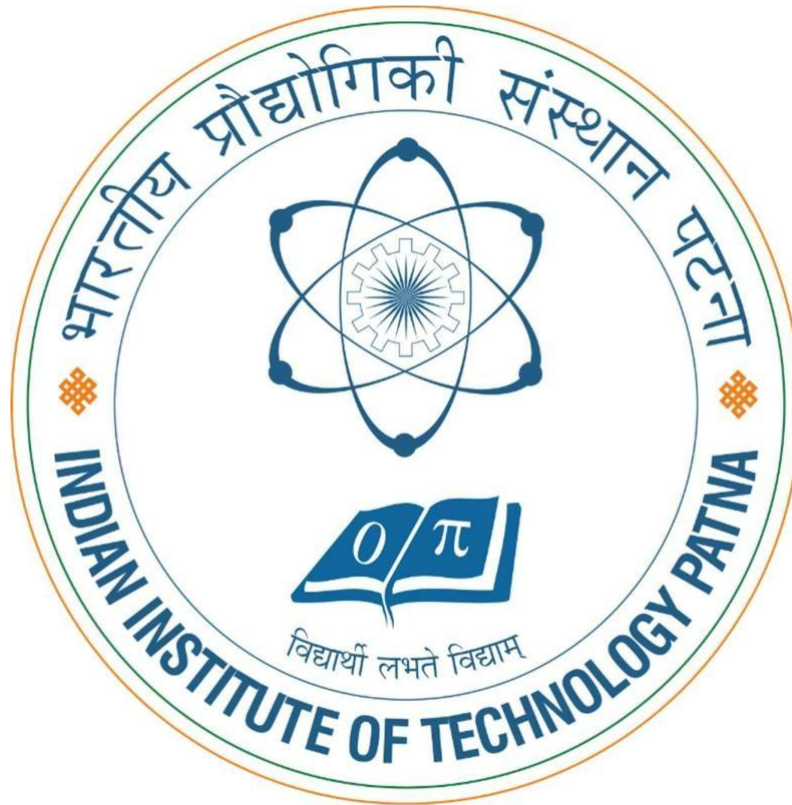


INDIAN INSTITUTE OF TECHNOLOGY PATNA
EC3202: DIGITAL SIGNAL PROCESSING LAB



Experiment No: 04

Topic: Discrete Fourier Transform & Fast Fourier Transform

Submitted By:

Name: Shreya Singh

Roll No: 2302VL10

Group No: 02

THEORY : -

An FFT computes the DFT and produces exactly the same result as evaluating the DFT definition directly; the most important difference is that an FFT is much faster. (In the presence of round-off error, many FFT algorithms are also much more accurate than evaluating the DFT definition directly, as discussed below.)

Let $x_0, \dots, x_{N-1}$ be complex numbers. The DFT is defined by the formula

$$X_k = \sum_{n=0}^{N-1} x_n \exp^{-i2\pi kn/N} \quad k = 0, 1, \dots, N-1$$

Evaluating this definition directly requires $O(N^2)$ operations: there are N outputs $X[k]$, and each output requires a sum of N terms. An FFT is any method to compute the same results in $O(N \log N)$ operations.

Aim 1: Find the DFT and IDFT of the given sequence using definitions. Say, $x[n] = \{0, 1, 2, 3\}$

CODE AND RESULT:-

```
clc;clear; close
all;

% Given sequence
x = [0 1 2 3]; N
= length(x);

% Preallocate DFT and IDFT
X = zeros(1, N); x_rec =
zeros(1, N);

%% DFT using definition
for k = 0:N-1      for n
= 0:N-1
    X(k+1) = X(k+1) + x(n+1)*exp(-1j*2*pi*k*n/N);
end end
disp('DFT X[k] = '); disp(X);

%% IDFT using definition
for n = 0:N-1
    for k = 0:N-1
        x_rec(n+1) = x_rec(n+1) + X(k+1)*exp(1j*2*pi*k*n/N);
    end
    x_rec(n+1) = x_rec(n+1)/N;
end
disp('Reconstructed sequence using IDFT = '); disp(x_rec);
SOLUTION:-
```

```
DFT X[k] =
    6.0000 + 0.0000i    -2.0000 + 2.0000i    -2.0000 - 0.0000i    -2.0000 - 2.0000i
```

Reconstructed sequence using IDFT =
0.0000 - 0.0000i 1.0000 - 0.0000i 2.0000 - 0.0000i 3.0000 + 0.0000i

Aim 2: Repeat **Aim 1** using compact matrix representation.

CODE AND RESULT:-

```
clc; clear;
close all;

% Given sequence
x = [0 1 2 3].'; % Column vector
N = length(x);

% Initialize DFT matrix
W = zeros(N, N);

% Construct DFT matrix using for loops
for k = 0:N-1      for n = 0:N-1
    W(k+1, n+1) = exp(-1j*2*pi*k*n/N);
end end
disp('DFT Matrix W = '); disp(W);

% DFT using matrix multiplication
X = W * x;
disp('DFT X[k] = '); disp(X.');

% IDFT using matrix method x_rec
= (1/N) * conj(W).' * X;

disp('Reconstructed sequence using IDFT = '); disp(x_rec.');
```

SOLUTION:-

DFT Matrix W =

```
1.0000 + 0.0000i  1.0000 + 0.0000i  1.0000 + 0.0000i  1.0000 + 0.0000i
1.0000 + 0.0000i  0.0000 - 1.0000i -1.0000 - 0.0000i -0.0000 + 1.0000i  1.0000
+ 0.0000i -1.0000 - 0.0000i  1.0000 + 0.0000i -1.0000 - 0.0000i
1.0000 + 0.0000i -0.0000 + 1.0000i -1.0000 - 0.0000i  0.0000 - 1.0000i
```

DFT X[k] =

```
6.0000 + 0.0000i -2.0000 + 2.0000i -2.0000 - 0.0000i -2.0000 - 2.0000i
```

Reconstructed sequence using IDFT =

```
0.0000 - 0.0000i  1.0000 - 0.0000i  2.0000 - 0.0000i  3.0000 + 0.0000i
```

Aim 4: To find N point FFT and IFFT of the given sequence (Decimation in time and decimation in frequency algorithms) without using the MATLAB inbuilt functions. Plot and check your results with the inbuilt functions.

CODE AND RESULT:-

```
clc; clear;
close all;

% Given sequence
x = [0 1 2 3]; N
= length(x);
stages = log2(N);

%% ===== BIT REVERSAL =====
br = zeros(1,N); bits = log2(N);
for n = 0:N-1
    r = 0;    temp =
n;    for i =
1:bits
        r = bitshift(r,1) + bitand(temp,1);
temp = bitshift(temp,-1);    end
br(n+1) = r + 1; end

%% ===== FFT USING DIT =====
X_dit = zeros(1,N); for
i = 1:N
    X_dit(i) = x(br(i));
end
for stage = 1:stages
    m = 2^stage;    half
= m/2;
    Wm = exp(-1j*2*pi*(0:half-1)/m);

    for k = 1:m:N
    for j = 0:half-1
        t = Wm(j+1) * X_dit(k+j+half);
u = X_dit(k+j);    X_dit(k+j) = u + t;
        X_dit(k+j+half) = u - t;    end    end end
%% ===== FFT USING DIF =====
X_dif = x;
for stage = stages:-1:1
    m = 2^stage;    half =
m/2;
    Wm = exp(-1j*2*pi*(0:half-1)/m);
    for k = 1:m:N    for j
= 0:half-1
        u =
X_dif(k+j);    v =
X_dif(k+j+half);
        X_dif(k+j) = u + v;
        X_dif(k+j+half) = (u - v) * Wm(j+1);
    end    end end

X_dif = X_dif(br);

%% ===== IFFT USING DIT =====
Xc = conj(X_dif);
Y_dif = Xc(br); for
stage = 1:stages
```

```

m = 2^stage;
half = m/2;
Wm = exp(-1j*2*pi*(0:half-1)/m);
    for k = 1:m:N        for j = 0:half-
1          t = Wm(j+1) *
Y_dit(k+j+half);          u =
Y_dit(k+j);          Y_dit(k+j) = u +
t;          Y_dit(k+j+half) = u - t;
end        end end
x_ifft_dit = conj(Y_dit)/N;

%% ===== IFFT USING DIF =====
Xc = conj(X_dit);
Y_dif = Xc;
for stage = stages:-1:1
m = 2^stage;    half =
m/2;
    Wm = exp(-1j*2*pi*(0:half-1)/m);
        for k = 1:m:N        for j
= 0:half-1          u =
Y_dif(k+j);          v =
Y_dif(k+j+half);
        Y_dif(k+j) = u + v;
        Y_dif(k+j+half) = (u - v) * Wm(j+1);
end        end end

Y_dif = Y_dif(br); x_ifft_dif
= conj(Y_dif)/N;

%% ===== VERIFICATION =====
X_builtin = fft(x); x_builtin = ifft(X_builtin);

%% ===== DISPLAY =====
disp('FFT using DIT = '); disp(X_dit); disp('FFT
using DIF = '); disp(X_dif); disp('IFFT using DIT
= '); disp(real(x_ifft_dit)); disp('IFFT using DIF
= '); disp(real(x_ifft_dif)); disp('Built-in FFT =
'); disp(X_builtin); disp('Built-in IFFT = ');
disp(real(x_builtin));

FFT using DIT =
6.0000 + 0.0000i -2.0000 + 2.0000i -2.0000 + 0.0000i -2.0000 - 2.0000i

FFT using DIF =
6.0000 + 0.0000i -2.0000 + 2.0000i -2.0000 + 0.0000i -2.0000 - 2.0000i

IFFT using DIT =
0    1    2    3

IFFT using DIF =
0    1    2    3

Built-in FFT =
6.0000 + 0.0000i -2.0000 + 2.0000i -2.0000 + 0.0000i -2.0000 - 2.0000i

```

Built-in IFFT =
0 1 2 3

Aim 3: Consider **Aim 1** and **Aim 2** and write "C" program to verify using Code Composer Studio (CCS-3.1) IDE and the DSP TMS 320C6713 DSK Kit.

```
#include <stdio.h>
#include <math.h>

#define N 4
#define PI 3.14159265358979323846

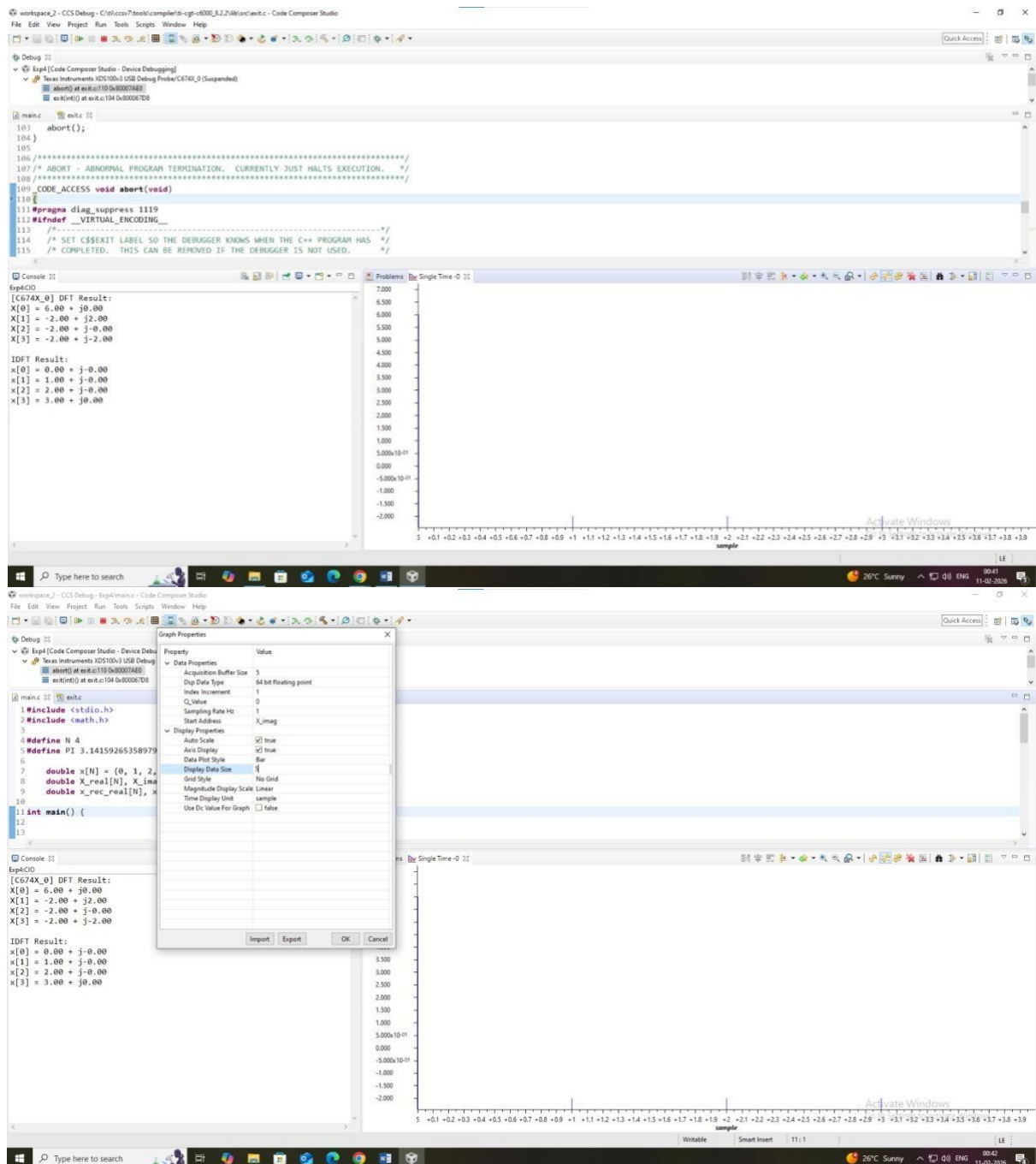
double x[N] = {0, 1, 2, 3};
double X_real[N], X_imag[N];      double
x_rec_real[N], x_rec_imag[N];

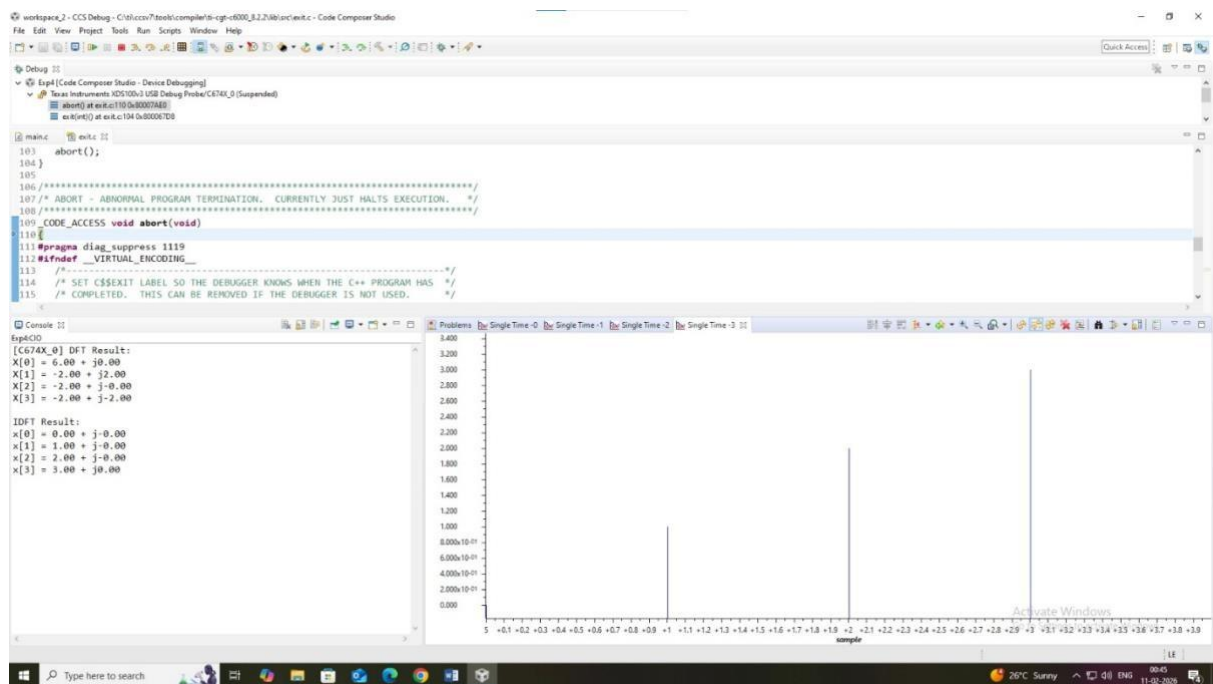
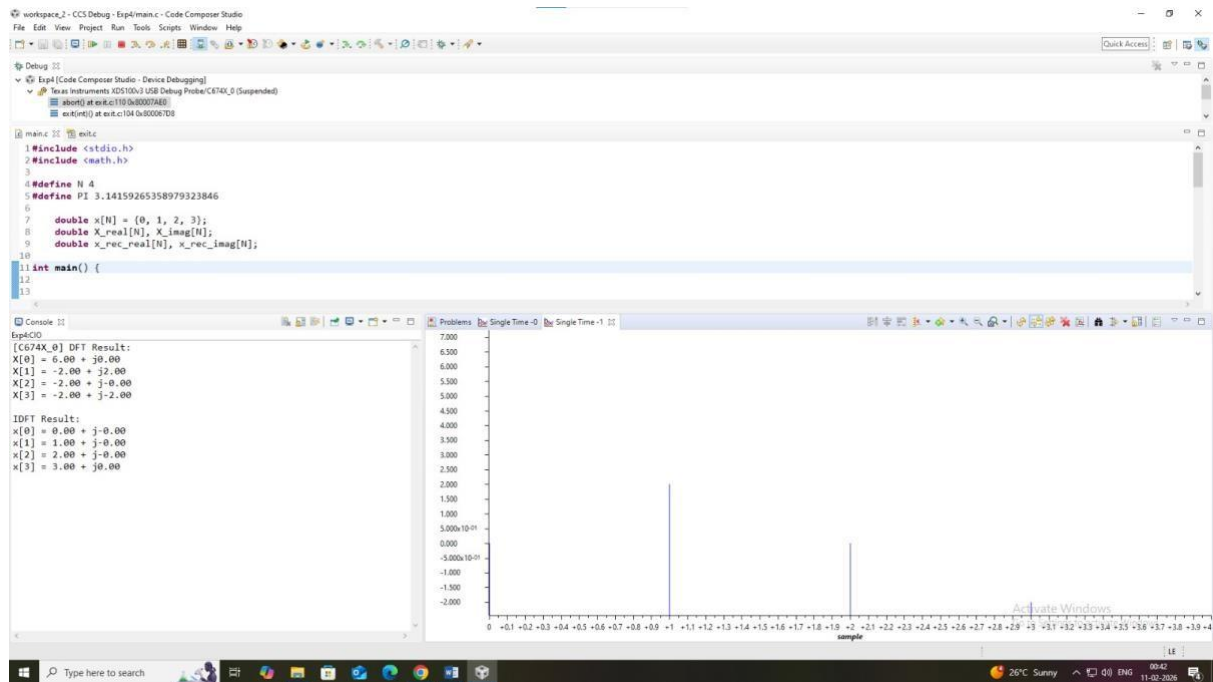
int main() {

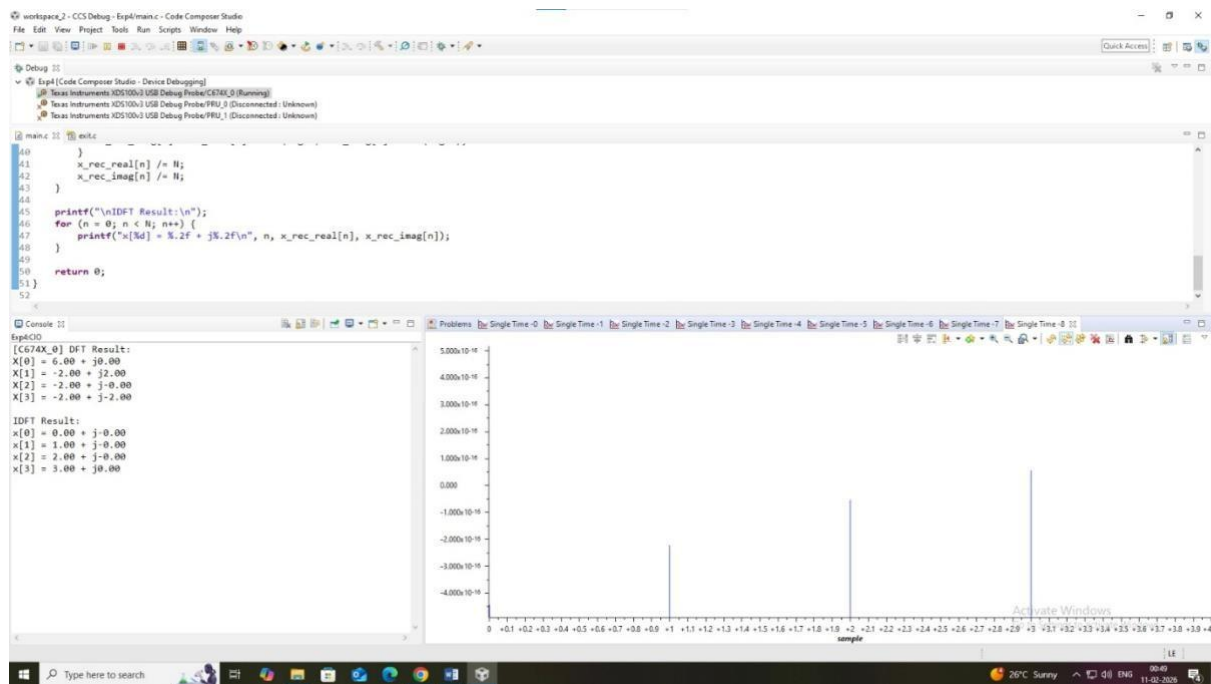
    int n, k;

    // Compute DFT
    for (k = 0; k < N; k++) {
        X_real[k] = 0;
        X_imag[k] = 0;
        for (n = 0; n < N; n++) {
            double angle = -2 * PI * k * n / N;
            X_real[k] += x[n] * cos(angle);
            X_imag[k] += x[n] * sin(angle);
        }
        printf("DFT Result:\n");
    }
    for (k = 0; k < N; k++) {
        printf("X[%d] = %.2f + j%.2f\n", k, X_real[k], X_imag[k]);
    }

    // Compute IDFT
    for (n = 0; n < N; n++) {
        x_rec_real[n] = 0;
        x_rec_imag[n] = 0;
        for (k = 0; k < N; k++) {
            double angle = 2 * PI * k * n / N;
            x_rec_real[n] += X_real[k] * cos(angle) - X_imag[k] * sin(angle);
            x_rec_imag[n] += X_real[k] * sin(angle) + X_imag[k] * cos(angle);
        }
        x_rec_real[n] /= N;
        x_rec_imag[n] /= N;
    }
    printf("\nIDFT Result:\n");
    for (n = 0; n < N; n++) {
        printf("x[%d] = %.2f + j%.2f\n", n, x_rec_real[n], x_rec_imag[n]);
    }
    return 0;
}
```







Aim 5: Consider **Aim 4** and write "C" program to verify using Code Composer Studio (CCS3.1) IDE and the DSP TMS 320C6713 DSK Kit.

```

#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979323846
#define N 4 // Defined specifically for the 4-point sequence

// Buffers for Real and Imaginary parts
float x_real[N] = {1.0, 2.0, 3.0, 4.0}; // The input sequence x(n)
float x_imag[N] = {0.0, 0.0, 0.0, 0.0}; // Imaginary part is 0 for real input
// Function to reverse bits for the DIT algorithm
int bit_reverse(int n, int bits) {    int
reversed_n = 0;
    int i;
    for (i = 0; i < bits; i++) {
        reversed_n = (reversed_n << 1) | (n & 1);
        n >>= 1;
    }
    return reversed_n;
}

// FFT Function
void fft(float *real, float *imag, int n) {
    int log2n = 0;    int temp_n = n;
    while (temp_n > 1) {    temp_n >>= 1;
        log2n++;
    }

    // Bit Reversal Permutation
    int i;

```

```

    for (i = 0; i < n; i++) {
int rev = bit_reverse(i, log2n);
if (i < rev) {
    float temp_r
= real[i];
    float temp_i =
imag[i];
    real[i] =
real[rev];
    imag[i] =
imag[rev];
    real[rev] =
temp_r;
    imag[rev] = temp_i;
}
}

// Butterfly Operations
int stage, k;
for (stage = 1; stage <= log2n; stage++) {
    int m = 1 << stage; // Size of the butterfly group
    int m2 = m >> 1; // Half size
float theta = -2.0 * PI / m;
float
w_real = 1.0;
float w_imag = 0.0;

    // Twiddle Factor increments
float wp_real = cos(theta);
float
wp_imag = sin(theta);
    int
j;
    for (j = 0; j < m2; j++) {
for (k = j; k < n; k += m) {
int t = k + m2;
        float u_real = w_real * real[t] - w_imag * imag[t];
        float u_imag = w_real * imag[t] + w_imag * real[t];

        real[t] = real[k] - u_real;
imag[t] = imag[k] - u_imag;
        real[k]
= real[k] + u_real;
        imag[k] =
imag[k] + u_imag;
    }
    // Update W
float temp_w = w_real;
        w_real = w_real * wp_real - w_imag * wp_imag;
w_imag = temp_w * wp_imag + w_imag * wp_real;
    }
}

void main() { //
Perform FFT    fft(x_real,
x_imag, N);

    // Infinite loop to hold the program for CCS viewing
while(1);
}

```

RESULT :-

